

ARTI 404 – Image Processing

Lab 3: Image Manipulations using OpenCV

Student: Abdullah Hejji ALnuwaysir

Student ID: 2240005501

Lab: 3

Import Libraries and Read the Image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

Path("images").mkdir(exist_ok=True)

# Read the input image
img = cv2.imread("images/input.jpg")

if img is None:
    raise FileNotFoundError("Could not find images/input.jpg")

plt.figure(figsize=(6, 6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Task 1 – Geometric Transformations

1. Increase the size of the image

```
# Increase the size of the image
scale = 2

resized = cv2.resize(
    img,
    None,
    fx=scale,
    fy=scale,
    interpolation=cv2.INTER_LINEAR
)

cv2.imwrite("images/resized.jpg", resized)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(resized, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")
plt.show()
```

2. Rotate the image by 120 degrees

```
# Rotate the image by 120 degrees
height, width = img.shape[:2]
center = (width // 2, height // 2)

rotation_matrix = cv2.getRotationMatrix2D(center, 120, 1.0)
rotated = cv2.warpAffine(img, rotation_matrix, (width, height))

cv2.imwrite("images/rotated_120.jpg", rotated)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image - 120 Degrees")
plt.axis("off")
plt.show()
```

3. Perform Shear transformation

```
# Shear the image
height, width = img.shape[:2]
shear_factor = 0.5

shear_matrix = np.float32([
```

```

        [1, shear_factor, 0],
        [0, 1, 0]
    ])

    new_width = int(width + shear_factor * height)
    sheared = cv2.warpAffine(img, shear_matrix, (new_width, height))

    cv2.imwrite("images/sheared.jpg", sheared)

    plt.figure(figsize=(8, 6))
    plt.imshow(cv2.cvtColor(sheared, cv2.COLOR_BGR2RGB))
    plt.title("Sheared Image")
    plt.axis("off")
    plt.show()

```

Task 2 – Intensity Transformations

1. Create a Negative Image

```

# Negative image
negative = 255 - img

cv2.imwrite("images/negative.jpg", negative)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(negative, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis("off")
plt.show()

```

2. Log transformation

```

# Log transformation
img_float = img.astype(np.float32)
c = 255 / np.log(1 + np.max(img_float))
log_transform = c * np.log(1 + img_float)
log_transform = np.uint8(np.clip(log_transform, 0, 255))

cv2.imwrite("images/log_transform.jpg", log_transform)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(log_transform, cv2.COLOR_BGR2RGB))
plt.title("Log Transformation")
plt.axis("off")
plt.show()

```

3. Power-law transformation

```

# Power-law transformation
gamma = 0.5
normalized = img.astype(np.float32) / 255.0
power_law = np.power(normalized, gamma)
power_law = np.uint8(power_law * 255)

cv2.imwrite("images/power_law_gamma_0.5.jpg", power_law)

plt.figure(figsize=(8, 6))
plt.imshow(cv2.cvtColor(power_law, cv2.COLOR_BGR2RGB))
plt.title("Power-Law Transformation (Gamma = 0.5)")
plt.axis("off")
plt.show()

```

End of Lab 3 All required transformations are implemented by the notebook code.

Code Verification

The notebook contains implementations for image reading, resizing, 120-degree rotation, shearing, negative transformation, logarithmic transformation, and power-law transformation ($\gamma = 0.5$). The input-image check is included so the notebook reports a clear error if `images/input.jpg` is missing.